

How ML Really Works?

Linear Regression

In statistics, **linear regression** is a model that estimates the relationship between a scalar response (**dependent variable**) and one or more explanatory variables (regressor or **independent variable**). A model with exactly one explanatory variable is a simple linear regression; a model with two or more explanatory variables is a multiple linear regression.

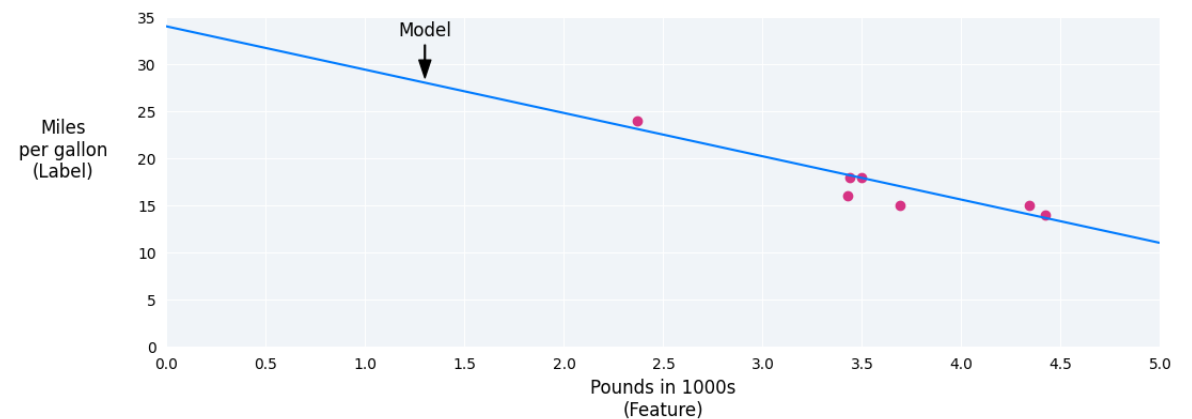
Real Life Applications:

- Predicting the Popularity of **Social Media** Posts
- Predicting the Price of **Residential** Properties
- Forecasting Sales Based on **Advertising** Spend
- Estimating **Product Demand** from Historical Data

For **example**, suppose we want to predict a car's fuel efficiency in miles per gallon based on how heavy the car is, and we have the following dataset:

Pounds in 1000s (feature)	Miles per gallon (label)
3.5	18
3.69	15
3.44	18
3.43	16
4.34	15
4.42	14
2.37	24

While plotting, we could create our own model by drawing a **best fit line** through the points:



Linear regression equation

In ML, we write the equation for a linear regression model as follows:

$$y' = b + w_1x_1$$

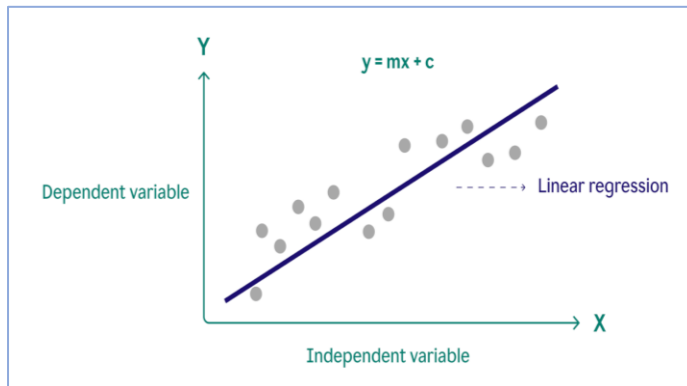
where :

y' is the dependent variable which is to be predicted.

x_1 is the independent variable

w_1 is the weight allotted (the slope of the line)

b is the bias term or the value of y when x is 0.



Steps in Linear Regression

- 1. Plot the Data Points:** Start by plotting the given data points, such as (1,5), (2,8), and (3,11).
- 2. Adjust the Line:** Draw a straight line and iteratively adjust its direction to minimise the distance (error) between the line and the data points.
- 3. Determine the Equation:** Once the line fits the data, identify the equation of the line.
- 4. Make Predictions:** Use the equation to predict values.

Example

In previous example, we'd calculate the weight and bias from the line we drew. The **bias** is 34 (where the line intersects the y-axis), and the **weight** is -4.6 (the slope of the line). We could use it to make predictions. For instance, using this model, a **4,000-pound car** would have a predicted fuel efficiency of **15.6 miles per gallon**.

Multiple Linear Regression

Multiple Linear Regression is a method that uses **two or more input variables** to predict a continuous output value.

For example, a model that predicts gas mileage could additionally use features such as the following:

- **Engine displacement**
- **Acceleration**
- **Number of cylinders**
- **Horsepower**

The equation looks like this:

$$y' = b + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5$$

Pounds Displacement Acceleration Number of cylinders Horsepower

Loss

Loss is a numerical metric that describes how wrong a model's predictions are. Loss measures the distance between the **model's predictions** and the **actual labels**. The goal of training a model is to **minimize the loss**, reducing it to its lowest possible value. This is done using **Gradient descent**, a technique that iteratively finds the weights and bias that produce the model with the lowest loss.

Loss Type	Definition
L1 loss	The sum of the absolute values of the difference between the predicted values and the actual values.
Mean absolute error (MAE)	The average of L_1 losses across a set of N examples.
L2 loss	The sum of the squared difference between the predicted values and the actual values.
Mean squared error (MSE)	The average of L_2 losses across a set of N examples

Python Implementation

Example – Predicting Body Mass Index(BMI) using Linear Regression

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_percentage_error
```

+ Code + Markdown

```
x = df[['Age', 'Height', 'Weight']]
y = df['Bmi']
```

```
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

LinearRegression
LinearRegression()

```
model.fit(X_train, y_train)
```

LinearRegression
LinearRegression()

```
pred = model.predict(X_test)
```

```
mean_absolute_percentage_error(y_test, pred)
```

```
0.027781091484666978
```

In Python, we can use a pre-built Linear Regression model using **Scikit Learn** library.

Steps :

Import the model -> Create a **LinearRegression()** object -> Use **.fit()** function to train the model on data ->

Store the **predictions** made on test data -> **Evaluate loss** using a suitable metric.

Resources to Learn More

[Google for Developers - Machine Learning Crash Course](#)

Interactive lessons, visualizations, exercises, and a dedicated Linear Regression module.

[Analytics Vidhya](#) : Beginner-friendly explanations and practical linear regression examples.

[Machine Learning Mastery](#) : Practical introduction covering the algorithm, representation, training, and data preparation.

[Scikit-learn Documentation](#) : Best for understanding how LinearRegression is implemented and used in Python.

YouTube Channels

[StatQuest with Josh Starmer](#) : Excellent for intuition and mathematical understanding.

[3Blue1Brown](#) : Great for developing mathematical intuition around linear algebra and regression.

[CampusX](#) : Strong Hindi explanations; their 100 Days of ML series includes Linear Regression intuition, mathematical formulation, coding, and regression metrics.